

Impact of Kernel Size and Stride on CNN Performance

Deep Learning

Amitha Mahesh
CB.AI.U4AID24105

Meghana Reddy M
CB.AI.U4AID24131

Nitheka A
CB.AI.U4AID24135

Shivvanii S
CB.AI.U4AID24146

1. Abstract

This project explores the impact of convolutional kernel size and stride on the performance of a Convolutional Neural Network (CNN) trained on the MNIST handwritten digit dataset. A simple single-layer CNN was implemented from scratch using NumPy and evaluated across nine configurations combining kernel sizes of 3×3 , 5×5 , and 7×7 with strides of 1, 2, and 3. Performance was assessed using validation accuracy, KL divergence, computational complexity (FLOPs), and training time. Results show that larger kernels with smaller strides generally achieve higher accuracy and lower divergence, with the 7×7 kernel and stride 1 configuration reaching the highest validation accuracy of 89.3%, though at higher computational cost. The study highlights the trade-off between model accuracy and computational efficiency when selecting convolution parameters in CNN design.

2. Problem Statement

Convolutional Neural Networks (CNNs) are widely used for image recognition tasks. Two important hyperparameters that influence convolutional layers are the kernel size and stride. Larger kernels capture broader spatial patterns, while larger strides reduce spatial resolution but improve computational efficiency. Both choices directly influence the computational complexity and training time of the network.

Despite their importance, there is limited systematic analysis of how these hyperparameters interact with respect to the following objectives:

- Maximising validation accuracy on a standard digit classification benchmark.
- Minimising KL divergence between predicted and true class distributions.
- Maintaining tractable computational complexity and training time.

This project addresses this gap by constructing a controlled experiment on the MNIST dataset, training nine CNN configurations under identical conditions, and analysing the resulting trade-offs.

3. Methodology

3.1 Dataset

The MNIST dataset contains 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels. Images were loaded from raw IDX files using a custom MnistDataloader class, and pixel values were normalized to the range [0,1] by dividing by 255.

To reduce computational cost, a subset of the dataset was used:

- **Training set:** 2,000 samples
- **Validation set:** 1,000 samples
- **Test set:** 1,000 samples

3.2 Network Architecture

A single-layer CNN was implemented entirely from scratch using NumPy. The architecture consists of:

- **Conv2D Layer:** 8 filters, variable kernel size (3, 5, or 7) and stride (1, 2, or 3). Kernels initialised with small random values.
- **ReLU Activation:** Applied element-wise after convolution to introduce non-linearity.
- **MaxPool2D Layer:** 2×2 pooling with stride 2, reducing spatial dimensions by half.
- **Flatten:** Converts 3D feature maps to a 1D vector for dense layers.
- **Dense Layer 1:** Fully connected layer with 128 neurons, followed by ReLU activation.
- **Dense Layer 2 (Output):** Fully connected layer with 10 neurons (one per class), followed by Softmax to produce class probabilities.

The network was trained using mini-batch gradient descent with a batch size of 32, a learning rate of 0.01, and cross-entropy loss over 10 epochs. Backpropagation was manually implemented for all layers.

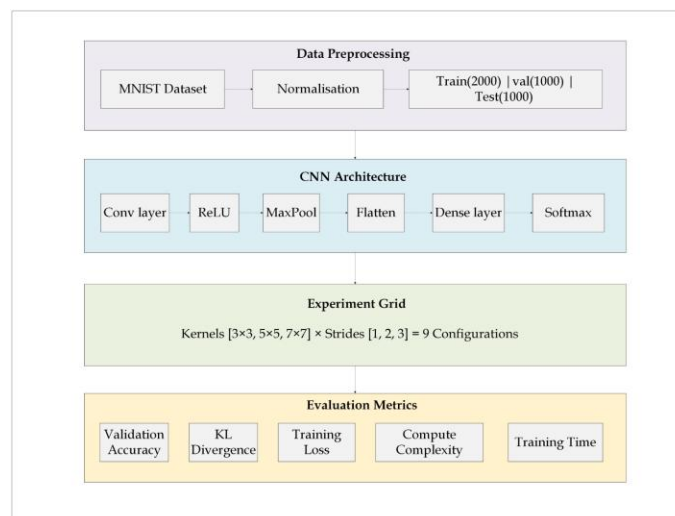


Figure 1: CNN model architecture and workflow

3.3 Evaluation Metrics

Four metrics were used to evaluate each configuration:

- **Validation Accuracy:** The proportion of correctly classified samples in the validation set.
- **KL Divergence:** Difference between predicted probability distribution and true labels.
- **Computational Complexity (FLOPs):** Estimated as: $\text{output_size}^2 \times \text{kernel_size}^2 \times \text{num_filters}$, capturing the relative cost of the convolutional operation.
- **Training Time (seconds):** Total runtime across all training epochs.

4. Results and Analysis

4.1 Summary of Results

Table 1 presents results across all nine kernel–stride configurations. The best-performing configuration is (k=7, s=1) with 89.3% validation accuracy with the lowest KL divergence of 0.382.

Kernel	Stride	Val Acc	KL Div	Complexity	Time (s)
3	1	0.8710	0.4269	48,672	2309.8
3	2	0.7980	0.6807	12,168	593.5
3	3	0.5130	1.7458	5,832	266.8
5	1	0.8640	0.4377	115,200	1989.1
5	2	0.8500	0.4870	28,800	522.1
5	3	0.6880	1.0918	12,800	222.0
7	1	0.8930	0.3823	189,728	1689.1
7	2	0.8730	0.4582	47,432	420.0
7	3	0.7310	0.8799	25,088	219.1

Table 1: Validation Accuracy, KL Divergence, Complexity and Training Time across all configurations

4.2 Effect on Validation Accuracy

Validation accuracy varies significantly across configurations. Stride 1 consistently produces the highest accuracy, while stride 3 leads to substantial performance degradation. The 7×7 kernel with stride 1 achieves the best accuracy (89.3%), followed by 3×3 stride 1 (87.1%) and 7×7 stride 2 (87.3%). Larger strides remove important spatial information, leading to poorer classification performance.

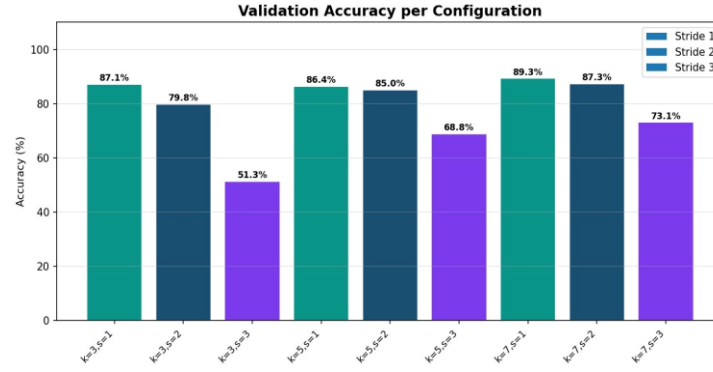


Figure 2: Validation Accuracy per Configuration (%)

The heatmap in Figure 3(a) further illustrates this relationship. Accuracy is consistently high (dark green) in the stride=1 column across all kernel sizes, and systematically decreases towards stride=3 (light yellow/white), confirming that stride has a stronger negative impact on accuracy than kernel size.

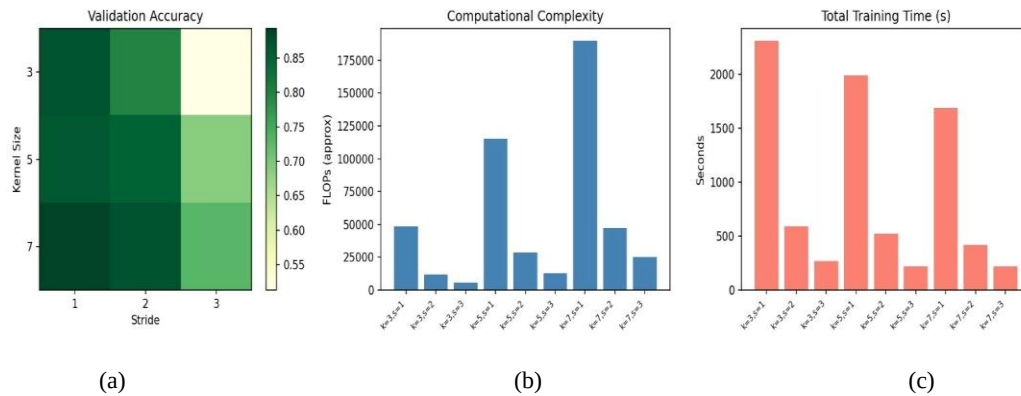


Figure 3: Heatmap of Validation Accuracy (a), Computational Complexity FLOPs (b), and Training Time in seconds (c)

4.3 Effect on KL Divergence

Figure 4 presents the KL divergence across different configurations with a KL = 0.5 threshold. Configurations with stride 1 (teal bars) remain below the threshold, indicating well-calibrated predictions.

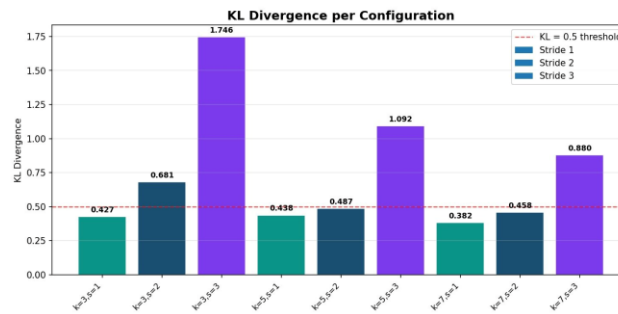


Figure 4: KL Divergence per Configuration with 0.5 threshold line

Stride 2 configurations (dark blue bars) show moderate divergence, generally close to the threshold. In contrast, stride 3 configurations (purple bars) exceed the threshold significantly, with the highest value observed at $k=3, s=3$ (1.746), indicating poor prediction reliability.

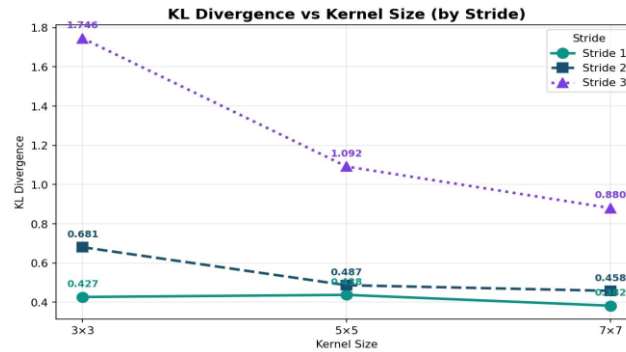


Figure 5: KL Divergence vs Kernel Size grouped by Stride

Figure 5 illustrates KL divergence across kernel sizes for different stride values. Stride 1 (teal line) maintains the lowest and most stable divergence across all kernel sizes. Stride 2 (dark blue line) shows moderate divergence that gradually decreases as kernel size increases. In contrast, stride 3 (purple line) exhibits the highest divergence, although it decreases noticeably from 3×3 to 7×7 . Despite this improvement, stride 3 remains significantly higher than stride 1 for all kernel sizes.

4.4 Accuracy vs. KL Divergence Trade-off

Figure 6 illustrates the relationship between validation accuracy and KL divergence. The top-left region represents the ideal zone, where accuracy is high and divergence is low. Most stride 1 configurations (teal points) and some stride 2 configurations (dark blue points) cluster near this region, with $k=7, s=1$ achieving the best position. In contrast, stride 3 configurations (purple points) appear in the bottom-right region, indicating lower accuracy and higher KL divergence, representing the least desirable performance.

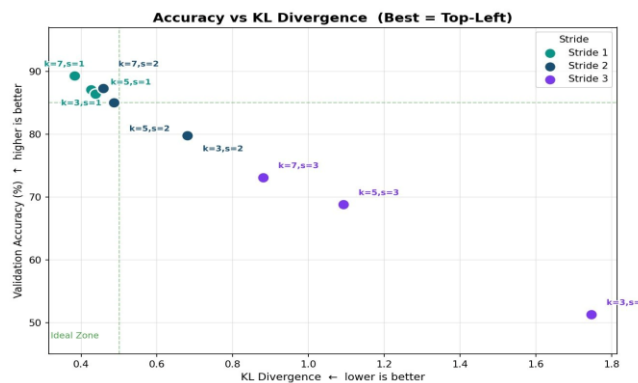


Figure 6: Accuracy vs KL Divergence Scatter Plot

4.5 Computational Complexity and Training Time

As shown in (b) and (c) of Figure 3, computational complexity (FLOPs) and training time are strongly correlated. The $k=7, s=1$ configuration is the most expensive at ~189,728 FLOPs and 1689 seconds, while $k=3, s=3$ is the cheapest at 5,832 FLOPs and 267 seconds. Notably, increasing stride is a very effective way to reduce cost: for $k=7$, moving from stride 1 to stride 3 reduces training time by ~87% (1689s $\rightarrow$ 219s) but causes accuracy to drop from 89.3% to 73.1%.

The best efficiency-accuracy trade-off is offered by $k=7, s=2$: it retains 87.3% accuracy with a KL divergence of 0.458 (below threshold), at 420 seconds — less than 25% of the cost of the full stride-1 configuration.

5. Discussion

This study shows that:

Results indicate that stride has a stronger impact on CNN performance than kernel size.

Configurations with stride 1 consistently achieve higher accuracy and lower KL divergence, while stride 3 significantly degrades performance due to the loss of spatial information caused by aggressive downsampling.

Kernel size plays a secondary role. Larger kernels (7×7) capture wider spatial context and provide slightly better accuracy and lower divergence compared to smaller kernels, but they also increase computational cost as FLOPs grow with kernel size.

Among the tested configurations, $k=7, s=2$ offers the best balance between accuracy and efficiency, achieving 87.3% accuracy with KL divergence of 0.458 while significantly reducing training time compared to **$k=7, s=1$** .

This study used a small subset of MNIST (2,000 training samples) and a single convolutional layer, so results may vary with larger datasets or deeper architectures. Future work could explore multi-layer CNNs, more complex datasets such as CIFAR-10, and optimized implementations to further analyze convolution hyperparameters.